

Лабораторна робота №4.

Нейронні мережі та згорткові архітектури на MNIST

25 серпня 2026 р.

Мета

Реалізувати багатошаровий перцептрон двічі — власноруч на NumPy зі своїм зворотним поширенням і той самий у PyTorch — та довести їхню чисельну еквівалентність; далі побудувати згорткову мережу й виміряти, скільки насправді дає кожен інгредієнт сучасного рецепта: архітектура, ініціалізація, нормалізація та аугментація.

Теоретичні відомості

Нейронна мережа — це композиція лінійних перетворень і нелінійностей, навчена градієнтним спуском. Уся практична складність зосереджена не у формулах, а в тому, що саме робить навчання стабільним: масштаб початкових ваг, вибір активації, нормалізація активацій усередині мережі.

- **Зворотне поширення.** Градієнт за всіма параметрами рахується одним проходом назад за ланцюговим правилом, тому коштує приблизно стільки ж, скільки прямий прохід. Реалізацію завжди перевіряють скінченними різницями: помилка в градієнті виглядає як «модель погано навчається» і легко приймається за невдалий темп навчання.
- **Активації та зникаючий градієнт.** Похідна сигмоїди не перевищує $1/4$, тож у мережі з десяти шарів множник сягає 10^{-6} і перші шари майже не отримують сигналу. ReLU має похідну 1 на додатній півосі й цієї проблеми не створює, натомість може «вимикати» нейрони назавжди.
- **Згортковий шар.** Замість зв'язку кожного входу з кожним виходом застосовується невелике ядро в кожній позиції зображення. Кількість ваг $C_{out}(C_{in}K^2 + 1)$ не залежить від розміру зображення, а спільні ваги дають еквіваріантність до зсуву.

- **Регуляризація глибоких мереж.** Пакетна нормалізація стабілізує масштаби активацій усередині мережі, dropout заважає нейронам покладатися на конкретних сусідів, аугментація породжує нові приклади перетвореннями, що зберігають мітку. Кожен прийом дає свій внесок, і його треба міряти окремо.

Корисні посилання для ознайомлення:

- [Stanford CS231n: Convolutional Neural Networks for Visual Recognition](#) — конспекти лекцій із поясненням кожного шару;
- [Dive into Deep Learning](#) — підручник із виконуваним кодом на PyTorch;
- [PyTorch: Learn the Basics](#) — офіційний вступний курс;
- [Інтерактивна 3D-візуалізація згорткової мережі](#) — намайлюйте цифру й подивіться на активації всіх шарів;
- [Google Colab](#) — безкоштовний GPU, якщо немає CUDA-сумісної відеокарти.

Порядок виконання

1. Обрати фреймворк: `pytorch` або `tensorflow` (далі наведено назви для PyTorch). Створити оточення, встановити `torch`, `torchvision`, `numpy`, `matplotlib`, `scikit-learn` і перевірити, який пристрій доступний.
2. Завантажити MNIST через `torchvision.datasets.MNIST` і підтвердити розміри: 60 000 тренувальних і 10 000 тестових зображень 28×28 . Показати сітку випадкових зразків, розподіл класів і базову лінію — логістичну регресію на сирих пікселях.
3. Реалізувати на NumPy мережу $784 \rightarrow 128 \rightarrow 10$ з активацією ReLU, softmax і перехресною ентропією: прямий прохід, зворотне поширення й крок оновлення окремими функціями. Перевірити градієнт скінченими різницями і навести максимальну відносну розбіжність.
4. Зібрати ту саму архітектуру засобами `torch.nn`, скопіювати в неї початкові ваги реалізації з попереднього пункту й довести еквівалентність: після одного кроку оптимізації втрати обох реалізацій мають збігатися до похибки округлення.
5. Порівняти чотири конфігурації: дві активації (`sigmoid` і `relu`) на двох ініціалізаціях (Хе та Ксав'є) з однаковим бюджетом епох, темпом навчання й зерном. Побудувати криві навчання на одному графіку та пояснити, яка пара і чому навчається найшвидше.

6. Реалізувати згорткову мережу: `conv(32, 3x3) → ReLU → conv(64, 3x3) → ReLU → maxpool(2x2) → dropout → fc(128) → fc(10)`. Спершу порахувати кількість параметрів кожного шару вручну за формулою $C_{out}(C_{in}K^2 + 1)$ і форми тензора після кожного шару, потім звірити з `sum(p.numel() for p in model.parameters())`.
7. Порівняти чотири конфігурації згорткової мережі — без обох механізмів, лише з пакетною нормалізацією, лише з `dropout`, з обома — на двох темпах навчання. Навести таблицю з тестовою точністю й часом епохи.
8. На підвибірці з 5000 тренувальних зразків порівняти три режими аугментації: без неї, помірний `RandomAffine` (зсув до 2 пікселів, поворот до 12°) і надмірний (поворот до 180°). Пояснити, чому третій режим шкодить саме на цьому наборі.
9. Для двох розмірів тренувальної вибірки — 2000 і 60 000 — побудувати криві тренувальної й валідаційної втрати. Визначити, чи потрібна тут рання зупинка, і показати епоху, на якій вона спрацювала б.
10. Візуалізувати вивчені ядра першого згорткового шару та карти ознак для однієї цифри. Порівняти їх із вагами першого шару повнозв'язної мережі з пункту 3.
11. Звести всі моделі в одну таблицю «модель — параметри — час епохи — тестова точність — приріст до базової лінії». Зробити висновки про те, який прийом дав найбільший внесок.

Додаткові завдання (необов'язкові)

Ці пункти виходять за межі аудиторного часу й виконуються за бажанням; вони не впливають на основну оцінку, але зараховуються як бонус.

1. Розширити пункт 5 до повної сітки: чотири активації (`sigmoid`, `tanh`, `relu`, `gelu`) на двох ініціалізаціях — вісім конфігурацій на одному графіку.
2. Побудувати криві навчання для чотирьох розмірів вибірки (500, 2000, 10 000, 60 000) і показати, як зсувається точка, де рання зупинка стає потрібною.
3. Замінити `MaxPool2d` на згортку з кроком 2 і порівняти обидва варіанти за точністю та кількістю параметрів.

Контрольні запитання

1. Що станеться з багатошаровою мережею, якщо активацію замінити тотожною функцією?

2. Ви реалізували зворотне поширення, і модель не навчається. Як за одну перевірку відрізнити помилку в градієнті від невдалого темпу навчання?
3. Похідна сигмоїди не перевищує $1/4$. Яке число це дає для мережі з десяти шарів і що з цього впливає?
4. Скільки параметрів має згортковий шар із 64 фільтрами 3×3 над входом із 32 каналів, і чому це число не залежить від розміру зображення?
5. Чому підвибірка (pooling) не має навчуваних параметрів і що вона дає натомість?
6. Поворот цифри на 180° формально теж є перетворенням зображення. Чому його не можна використовувати для аугментації MNIST?
7. Мережа дала 95 % на MNIST. Чому цього числа замало, щоб назвати результат хорошим?